

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Fauzian Ramadhan - 5024241080

2026

1 Langkah-Langkah Percobaan

Sebelum memulai seluruh rangkaian percobaan pada Modul 1, disiapkan satu proyek utama pada ekstensi PlatformIO di dalam Visual Studio Code (misalnya diberi nama `modul1-praktikum`). Proyek tunggal ini digunakan secara berkesinambungan untuk Percobaan 1 hingga Percobaan 5 dengan cara memperbarui isi berkas konfigurasi `platformio.ini` serta berkas kode sumber pada direktori `src/` (`main.cpp` atau `main.c`) sesuai kebutuhan masing-masing percobaan. Setiap kali berkas `platformio.ini` diperbarui, proses *refresh/re-index* pada PlatformIO ditunggu hingga selesai sebelum menekan tombol *Build* dan *Upload*.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Proyek Modul 1 dibuka pada VSCode, kemudian isi berkas `platformio.ini` diatur dengan konfigurasi target *board* STM32 Blackpill (`blackpill_f401cc`), *framework* Arduino, dan protokol pengunggahan `stlink` sebagai berikut:

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Perangkat pemrogram ST-Link V2 dihubungkan ke pin header *Serial Wire Debug* (SWD) pada *board* STM32 Blackpill menggunakan empat utas kabel jumper *female-female* dengan pemetaan pin sebagai berikut:

   - Pin `SWDIO` ST-Link dihubungkan ke pin `SWDIO` Blackpill (jalur data/debug).
   - Pin `SWCLK` ST-Link dihubungkan ke pin `SWCLK` Blackpill (jalur *clock*/debug).
   - Pin `GND` ST-Link dihubungkan ke pin `GND` Blackpill (referensi *ground* bersama).
   - Pin `3.3V` ST-Link dihubungkan ke pin `3V3` Blackpill (catu daya tegangan target 3.3 V).

3. Setelah koneksi kabel dipastikan sesuai dan memakai tegangan 3.3 V, ST-Link dicolokkan ke port USB komputer.

4. Kode program *Blink* ditulis pada berkas `src/main.cpp` untuk mengendalikan LED *onboard* pada pin `PC13`:

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
```

```
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}
```

5. Tombol *Build* dan *Upload* pada bilah alat PlatformIO ditekan, kemudian diamati nyala-mati LED *onboard* (PC13) pada papan Blackpill yang berkedip setiap setengah detik (500 ms).

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Isi berkas `platformio.ini` diganti menjadi konfigurasi untuk *board* ESP32 DevKit v1 (`esp32dev`) dengan menggunakan *framework native* `espidf`, lalu ditunggu hingga proses indeksasi selesai:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Papan ESP32 DevKit dihubungkan langsung ke komputer menggunakan kabel USB, dan dipastikan port serial perangkat telah dikenali oleh sistem operasi.

3. Berkas `src/main.cpp` dari percobaan sebelumnya dihapus, lalu dibuat berkas baru bernama `src/main.c`.

4. Kode program *Blink* berbasis pustaka driver GPIO ESP-IDF dan FreeRTOS ditulis ke dalam `src/main.c`:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

5. Tombol *Build* dan *Upload* ditekan, kemudian diamati LED *onboard* ESP32 (`GPIO 2`) yang berkedip secara periodik setiap 500 ms serta dianalisis perbedaan struktur kodenya terhadap *framework* Arduino.

1.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Isi berkas `platformio.ini` diubah kembali menggunakan *framework* `arduino` untuk *board* `esp32dev`. Berkas `src/main.c` dihapus dan dibuat kembali berkas `src/main.cpp`:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

2. **Bagian A (Konfigurasi Eksternal):** Dua buah *pushbutton tactile* dan dua resistor 10 kΩ dirangkai pada *breadboard* dengan skema:

   - *Pushbutton* 1 (*pull-up* eksternal): Satu kaki dihubungkan ke `GPIO 32` sekaligus ke jalur `3.3V` melalui resistor 10 kΩ, sedangkan kaki pasangan seberangnya dihubungkan ke `GND`.
   - *Pushbutton* 2 (*pull-down* eksternal): Satu kaki dihubungkan ke `GPIO 33` sekaligus ke jalur `GND` melalui resistor 10 kΩ, sedangkan kaki pasangan seberangnya dihubungkan ke `3.3V`.

3. Kode Program A ditulis pada `src/main.cpp`, diunggah ke ESP32, lalu status pembacaan kedua tombol saat ditekan dan tidak ditekan diamati melalui *Serial Monitor* pada *baud rate* `115200`:

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor dipasang eksternal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor dipasang eksternal
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

4. **Bagian B (Konfigurasi Internal):** Kedua resistor fisik 10 kΩ dilepas dari *breadboard*, sementara kedua *pushbutton* tetap terpasang (*Pushbutton* 1 menghubungkan `GPIO 32` ke `GND`, dan *Pushbutton* 2 menghubungkan `GPIO 33` ke `3.3V`).

5. Isi `src/main.cpp` diganti dengan Kode Program B yang mengaktifkan resistor internal (`INPUT_PULLUP` dan `INPUT_PULLDOWN`), kemudian diunggah dan dibandingkan hasilnya pada *Serial Monitor* dengan hasil Bagian A:

```
#include <Arduino.h>

```

```
#define BUTTON_PULLUP_PIN 32   // pull-up internal
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);     // pull-up internal ESP32
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Sebuah *pushbutton tactile* dipasang pada *breadboard* dengan satu kaki terhubung ke `GPIO 25` ESP32 dan kaki lainnya terhubung ke `GND` (menggunakan konfigurasi `INPUT_PULLUP` internal).

2. Kode program pembanding pencacah tanpa *debouncing* (`counterNoDebounce`) dan dengan *debouncing* (`counterWithDebounce`) ditulis pada `src/main.cpp` dengan nilai awal `DEBOUNCE_DELAY` sebesar 50 ms:

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int reading = digitalRead(BUTTON_PIN);

  // ===== Tanpa Debouncing =====
```

```
  if (reading == LOW && rawLastState == HIGH) {
    counterNoDebounce++;
  }
  rawLastState = reading;

  // ===== Dengan Debouncing =====
  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
    }
  }
  lastFlickerableState = reading;

  // ===== Tampilkan kedua nilai secara bersamaan =====
  static unsigned long lastPrintedNoDebounce = 0, lastPrintedWithDebounce = 0;
  if (counterNoDebounce != lastPrintedNoDebounce || counterWithDebounce !=
    lastPrintedWithDebounce) {
    Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n",
    counterNoDebounce, counterWithDebounce);
    lastPrintedNoDebounce = counterNoDebounce;
    lastPrintedWithDebounce = counterWithDebounce;
  }
}
```

3. Program diunggah ke ESP32, lalu tombol ditekan dengan berbagai variasi gaya tekan (ditekan cepat dan tegas, kemudian ditekan pelan atau ragu-ragu) sambil mengamati selisih pertambahan nilai kedua penghitung pada *Serial Monitor*.

4. Nilai makro `DEBOUNCE_DELAY` diubah menjadi `10` (10 ms), diunggah ulang, dan diuji kembali.

5. Nilai makro `DEBOUNCE_DELAY` diubah menjadi `200` (200 ms), diunggah ulang, lalu tombol ditekan secara cepat berulang kali untuk mengamati perbedaan responsivitas dan efektivitas penyaringan getaran dari ketiga nilai waktu tunda (10 ms, 50 ms, dan 200 ms).

1.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Sensor LDR (*Light Dependent Resistor*) dan resistor tetap 10 kΩ dirangkai sebagai pembagi tegangan pada *breadboard*: satu kaki LDR dihubungkan ke `3.3V`, kaki LDR lainnya dihubungkan ke `GPIO 34` sekaligus dihubungkan ke `GND` melalui resistor 10 kΩ.

2. Kode program pembacaan nilai mentah ADC 12-bit dan konversi tegangannya ditulis pada berkas `src/main.cpp`:

```
#include <Arduino.h>

```

```
#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

3. Program diunggah ke ESP32 dan *Serial Monitor* dibuka pada *baud rate* 115200.

4. Permukaan sensor LDR ditutup dengan tangan (kondisi gelap) lalu disinari menggunakan cahaya lampu/senter (kondisi terang), kemudian dicatat perubahan nilai ADC mentah (rentang 0–4095) beserta nilai tegangan terukurnya.

2 Analisis Hasil Percobaan

2.1 Analisis Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Berdasarkan hasil Percobaan 1, program *Blink* berhasil dikompilasi dan diunggah ke mikrokontroler STM32F401CCU6 pada papan STM32 Blackpill menggunakan perangkat pemrogram eksternal ST-Link V2 melalui antarmuka protokol *Serial Wire Debug* (SWD). Penggunaan ST-Link V2 mutlak diperlukan karena papan Blackpill tidak menyertakan sirkuit *programmer/debugger* maupun konverter USB-to-Serial bawaan untuk memindahkan kode mesin secara langsung seperti pada papan devkit umumnya. Keempat jalur SWD memiliki peran krusial: **SWDIO** mentransmisikan data instruksi dua arah, **SWCLK** menyinkronkan detak komunikasi, **GND** menyamakan potensial referensi, dan **3.3V** menyuplai daya ke mikrokontroler.

Dari sisi perangkat lunak, penggunaan *framework* Arduino (STM32duino) di atas PlatformIO menyederhanakan akses perangkat keras secara signifikan. Tanpa perlu mengonfigurasi register **RCC** untuk mengaktifkan *clock* port C maupun mendefinisikan struktur **GPIO_InitTypeDef** seperti pada pustaka HAL murni, pemanggilan **pinMode(PC13, OUTPUT)** secara otomatis menginisialisasi periferal GPIO di balik layar. Fenomena penting yang teramati dan sesuai dengan teori rangkaian skematik Blackpill adalah sifat LED *onboard* pada pin **PC13** yang berlogika **aktif-*LOW*** (*current sinking*), di mana anoda LED terhubung ke jalur 3.3 V dan katodanya masuk ke pin **PC13**. Akibatnya, saat instruksi **digitalWrite(LED_PIN, LOW)** dieksekusi, pin **PC13** bernilai 0 V sehingga tercipta beda potensial yang mengalirkan arus dan menyalakan LED. Sebaliknya, saat diberi logika **HIGH** (3.3 V), tidak ada beda potensial antara anoda dan katoda sehingga LED padam.

2.2 Analisis Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Pada Percobaan 2, pemrograman mikrokontroler ESP32 dilakukan langsung melalui kabel USB tanpa memerlukan ST-Link karena papan ESP32 DevKit v1 telah mengintegrasikan *chip* jembatan konverter

USB-to-Serial bawaan (CP2102/CH340) yang menerjemahkan paket data USB dari PC menjadi sinyal serial UART ke pin **RX0**/**TX0** ESP32 beserta sirkuit *auto-reset bootloader*. Berbeda dengan pin **PC13** pada Blackpill, LED *onboard* pada ESP32 (**GPIO 2**) terhubung secara **aktif-*HIGH*** (*current sourcing*), sehingga pemberian level logika **1** melalui **gpio_set_level(LED_PIN, 1)** membuat LED menyala, dan logika **0** membuat LED padam.

Perbandingan antara Percobaan 1 (Arduino) dan Percobaan 2 (ESP-IDF) memperlihatkan perbedaan arsitektur perangkat lunak yang mendasar. Pada *framework* Arduino, struktur program diabstraksikan menjadi fungsi **setup()** dan **loop()**. Sebaliknya, pada *framework native* ESP-IDF yang dibangun langsung di atas sistem operasi waktu nyata **FreeRTOS**, berkas program ditulis dalam bahasa C (**main.c**) dengan titik masuk berupa fungsi **app_main(void)** yang dieksekusi sebagai sebuah *task* utama. Konfigurasi pin harus dilakukan secara eksplisit dengan mereset konfigurasi bawaan melalui **gpio_reset_pin()** lalu menetapkan arah pin melalui **gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT)**. Selain itu, penundaan waktu pada ESP-IDF menggunakan fungsi **vTaskDelay(pdMS_TO_TICKS(500))** yang bekerja berdasarkan *tick* penjadwalan (*scheduler*) FreeRTOS. Keunggulan utama mekanisme ini dibandingkan **delay(500)** biasa pada Arduino adalah sifatnya yang *non-blocking* terhadap CPU: selama periode tunda 500 ms berlangsung, *task* **app_main** memasuki status *Blocked* dan membebaskan sumber daya prosesor untuk mengeksekusi *task* latar belakang lainnya (seperti tumpukan protokol WiFi/Bluetooth atau *task* pengguna lain).

2.3 Analisis Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Hasil pengujian pada Percobaan 3 membuktikan bahwa baik konfigurasi resistor eksternal (Kode Program A dengan **pinMode(pin, INPUT)** dan resistor fisik 10 kΩ) maupun konfigurasi resistor internal (Kode Program B dengan **INPUT_PULLUP** dan **INPUT_PULLDOWN** tanpa resistor fisik) menghasilkan perilaku logika pembacaan yang identik dan stabil pada *Serial Monitor*.

- Pada **GPIO 32 (*Pull-up*)**, ketika tombol tidak ditekan, jalur pin ditarik ke tegangan 3.3 V melalui resistor (eksternal maupun internal) sehingga **digitalRead(32)** menghasilkan logika **HIGH** dan mencetak **"TIDAK DITEKAN"**. Saat tombol ditekan, pin **GPIO 32** terhubung langsung ke jalur **GND** (0 V) sehingga pembacaan turun menjadi **LOW** dan mencetak **"DITEKAN"** (logika *active-low*).
- Pada **GPIO 33 (*Pull-down*)**, ketika tombol tidak ditekan, jalur pin ditarik ke **GND** melalui resistor sehingga terbaca **LOW** (**"TIDAK DITEKAN"**). Saat tombol ditekan, pin terhubung langsung ke jalur catu 3.3 V sehingga terbaca **HIGH** (**"DITEKAN"**, logika *active-high*).

Keberadaan resistor *pull-up* dan *pull-down* ini terbukti efektif menghilangkan kondisi mengambang (*floating state*) saat kontak sakelar terbuka, sekaligus membatasi arus agar tidak terjadi hubung singkat (*short circuit*) antara jalur 3.3 V dan **GND** ketika tombol ditekan. Dari segi kepraktisan perancangan perangkat keras, pemanfaatan resistor internal ESP32 (≈ 45 kΩ) sangat menguntungkan karena menghemat jumlah komponen fisik pada *breadboard*/PCB. Namun, faktor penting yang wajib diperhatikan adalah batasan arsitektur pin ESP32, di mana pin khusus masukan (*input-only pins* pada **GPIO 34 hingga GPIO 39**) tidak memiliki rangkaian transistor/resistor *pull-up* maupun *pull-down* internal di dalam silikonnya. Oleh karena itu, pemilihan **GPIO 32** dan **GPIO 33** pada percobaan ini sudah tepat karena keduanya merupakan pin GPIO dua arah penuh yang mendukung *pull* internal.

2.4 Analisis Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Pada Percobaan 4, pengamatan pada *Serial Monitor* menunjukkan perbedaan mencolok antara variabel **counterNoDebounce** dan **counterWithDebounce**, terutama saat tombol ditekan secara pelan/ragu-ragu atau ditekan dengan keras. Ketika tombol fisik ditekan satu kali, nilai **counterNoDebounce** sering kali melonjak bertambah 2 hingga 5 angka sekaligus, sedangkan **counterWithDebounce** (pada **DEBOUNCE_DELAY = 50**) selalu bertambah secara konsisten tepat sebesar 1. Lonjakan pada **counterNoDebounce** terjadi karena lempeng logam pegas di dalam *tactile pushbutton* memantul secara mekanis saat bersentuhan, menciptakan rentetan transisi tegangan **HIGH**$\rightarrow$**LOW** dalam orde beberapa milidetik. Karena fungsi **loop()** dieksekusi oleh CPU Xtensa LX6 dengan frekuensi sangat tinggi, setiap transisi pantulan langsung dihitung sebagai penekanan baru oleh blok kode tanpa *debouncing*.

Sebaliknya, algoritma *software debouncing* bekerja sebagai penyaring lolos-rendah berbasis waktu (*time-domain filter*): setiap kali terdeteksi perubahan status mentah (**reading != lastFlickerableState**), pencatat waktu **lastDebounceTime** direset ke waktu **millis()** saat itu. Status tombol yang sah (**stableState**) baru diperbarui apabila tidak ada lagi perubahan sinyal selama durasi yang melampaui **DEBOUNCE_DELAY**. Berdasarkan pengujian variasi nilai waktu tunda, diperoleh analisis kompromi (*trade-off*) kinerja sebagai berikut:

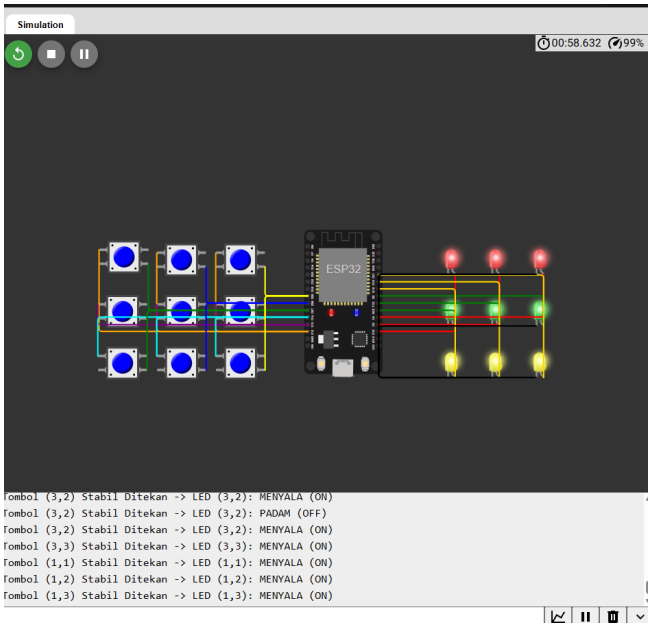
- **DEBOUNCE_DELAY = 10 ms:** Waktu tunggu sangat singkat sehingga tombol terasa sangat responsif, namun ketika tombol ditekan secara pelan atau mulai aus, getaran mekanik yang berlangsung lebih dari 10 ms masih sempat lolos dan menyebabkan **counterWithDebounce** sesekali bertambah lebih dari 1.
- **DEBOUNCE_DELAY = 50 ms:** Merupakan titik keseimbangan (*sweet spot*) paling optimal untuk interaksi manusia. Seluruh getaran pantulan mekanik berhasil diredam sepenuhnya tanpa menimbulkan jeda yang terasa oleh jari pengguna.
- **DEBOUNCE_DELAY = 200 ms:** Sangat kebal terhadap getaran *bouncing*, tetapi ketika pengguna menekan tombol secara cepat berulang kali (lebih cepat dari 5 kali per detik), banyak penekanan yang diabaikan oleh program karena durasi tahan tombol belum mencapai ambang 200 ms, sehingga sistem terasa lambat dan tidak responsif.

2.5 Analisis Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Pada pelaksanaan praktikum, Percobaan 5 tidak sempat diujicobakan secara langsung oleh kelompok karena keterbatasan waktu sesi praktikum yang habis saat menyelesaikan Percobaan 1 hingga Percobaan 4. Kendala utama disebabkan oleh terjadinya *error* pada ekstensi PlatformIO di laptop yang digunakan pada awal praktikum ketika proses kompilasi dan indeksasi *framework* berlangsung, sehingga praktikan harus berganti menggunakan laptop anggota kelompok yang lain. Proses instalasi ulang serta konfigurasi awal (*set-up toolchain* dan pengunduhan paket *platform/framework*) pada laptop pengganti tersebut memakan waktu yang cukup lama hingga sesi berakhir. Meskipun demikian, secara teoretis pada Percobaan 5 ini pin **GPIO 34** (kanal **ADC1**) berfungsi membaca tegangan kontinu dari titik tengah rangkaian pembagi tegangan antara sensor LDR dan resistor 10 kΩ melalui

fungsi **analogRead(34)** dengan resolusi 12-bit $(0–4095)$, di mana saat kondisi gelap resistansi LDR membesar sehingga nilai ADC dan tegangan terukur turun mendekati 0, sedangkan saat kondisi terang resistansi LDR mengecil sehingga nilai ADC naik mendekati 4095 $(3.3$ V).

3 Hasil Tugas Modul



Gambar 1: Matrix Pushbutton 3x3

Pada tugas modul ini dirancang simulasi sistem **Matrix Pushbutton 3x3** yang mengendalikan **Matrix LED 3x3** berbasis mikrokontroler ESP32 menggunakan *framework* Arduino pada simulator Wokwi. Setiap tombol pada koordinat (baris, kolom) mengendalikan status nyala/padam (*toggle*) LED pada koordinat (baris, kolom) yang bersesuaian dengan menerapkan algoritma *software debouncing* (waktu tunda 50 ms) pada masing-masing tombol.

3.1 Skema Pin dan Kode Program Simulasi

Jalur baris tombol matriks dihubungkan ke pin keluaran **GPIO 13, 12, dan 14**, sedangkan jalur kolom tombol dihubungkan ke pin masukan **GPIO 27, 26, dan 25** dengan konfigurasi **INPUT_PULLUP** internal. Kesembilan LED pada matriks 3x3 dihubungkan ke pin keluaran **GPIO 15, 2, 4** (Baris 1), **GPIO 16, 17, 5** (Baris 2), serta **GPIO 18, 19, 21** (Baris 3). Berikut adalah kode program lengkap yang diimplementasikan:

```
#include <Arduino.h>

#define ROWS 3
#define COLS 3
#define DEBOUNCE_DELAY 50

const int rowPins[ROWS] = {13, 12, 14};
const int colPins[COLS] = {27, 26, 25};

const int ledPins[ROWS][COLS] = {
```

```
  {15, 2, 4},
  {16, 17, 5},
  {18, 19, 21}
};

int lastFlickerableState[ROWS][COLS];
int stableState[ROWS][COLS];
unsigned long lastDebounceTime[ROWS][COLS];
bool ledState[ROWS][COLS];

void setup() {
  Serial.begin(115200);
  for (int r = 0; r < ROWS; r++) {
    pinMode(rowPins[r], OUTPUT);
    digitalWrite(rowPins[r], HIGH);
  }
  for (int c = 0; c < COLS; c++) {
    pinMode(colPins[c], INPUT_PULLUP);
  }
  for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
      pinMode(ledPins[r][c], OUTPUT);
      digitalWrite(ledPins[r][c], LOW);
      lastFlickerableState[r][c] = HIGH;
      stableState[r][c] = HIGH;
      lastDebounceTime[r][c] = 0;
      ledState[r][c] = false;
    }
  }
}

void loop() {
  for (int r = 0; r < ROWS; r++) {
    digitalWrite(rowPins[r], LOW);
    delayMicroseconds(10);

    for (int c = 0; c < COLS; c++) {
      int reading = digitalRead(colPins[c]);

      if (reading != lastFlickerableState[r][c]) {
        lastDebounceTime[r][c] = millis();
      }

      if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
        if (reading != stableState[r][c]) {
          stableState[r][c] = reading;
          if (stableState[r][c] == LOW) {
            ledState[r][c] = !ledState[r][c];
            digitalWrite(ledPins[r][c], ledState[r][c] ? HIGH : LOW);
            Serial.printf("Tombol (%d,%d) Ditekan -> LED (%d,%d): %s\n",
                          r + 1, c + 1, r + 1, c + 1,
                          ledState[r][c] ? "ON" : "OFF");
          }
        }
```

```
65        }
66        lastFlickerableState[r][c] = reading;
67      }
68      digitalWrite(rowPins[r], HIGH);
69    }
70  }
```

3.2 Jawaban Pertanyaan Analisis Tugas Modul

1. **Urgensi Debouncing pada Matrix Pushbutton 3x3:**
   Debouncing tetap mutlak diperlukan meskipun pada satu waktu hanya satu tombol yang ditekan, karena fenomena *bouncing* berasal dari sifat mekanis lempeng logam pegas di dalam masing-masing *pushbutton* saat kontak menyentuh atau terlepas, bukan akibat penekanan banyak tombol secara bersamaan. Tumbukan elastis antar-kontak logam menimbulkan transisi logika *HIGH–LOW* berulang kali dalam waktu beberapa milidetik sebelum kontak benar-benar stabil. Karena pemindaian baris-kolom (*matrix scanning*) pada ESP32 berlangsung sangat cepat (ribuan siklus per detik), satu kali penekanan fisik tanpa *debouncing* akan terdeteksi sebagai beberapa kali transisi penekanan beruntun sehingga LED pada posisi (baris, kolom) tersebut akan berkedip atau berubah status (*toggle*) secara keliru. Penerapan *debouncing* (50 ms) memastikan status LED hanya diperbarui satu kali setelah pembacaan sinyal tombol benar-benar stabil.

2. **Analisis Konfigurasi Pull-up pada Kolom Tombol dan Bahaya Kondisi Floating:**
   Pada rancangan ini, pin pembacaan kolom (**GPIO 27, 26, dan 25**) dikonfigurasi menggunakan **resistor *pull-up* internal (INPUT_PULLUP)**, sedangkan pin baris (**GPIO 13, 12, dan 14**) diatur sebagai **OUTPUT** yang dipindai bergiliran dengan logika *LOW*. Saat tombol **tidak ditekan**, sirkuit tombol terbuka dan resistor *pull-up* internal menarik jalur kolom ke tegangan 3.3 V sehingga pin kolom terbaca stabil pada logika ***HIGH***. Saat tombol pada baris yang sedang aktif (*LOW*) ditekan, jalur kolom terhubung ke baris tersebut sehingga terbaca ***LOW***. Kondisi pin kolom ini tidak boleh dibiarkan *floating* (tanpa *pull-up*/*pull-down*) karena saat tombol terbuka, jalur kawat kolom yang membentang pada matriks tidak memiliki referensi tegangan dan berimpedansi sangat tinggi. Jalur tersebut akan bertindak seperti antena yang menangkap derau (*noise*) elektromagnetik maupun induksi silang (*crosstalk*) dari sinyal pemindaian baris, sehingga nilai pembacaan kolom berubah-ubah secara acak antara *HIGH* dan *LOW* serta menyalakan LED secara liar tanpa adanya penekanan tombol.

3. **Perbedaan GPIO sebagai Digital Input (Tombol) vs Analog Input (ADC dengan LDR):**
   Perbedaan mendasar terletak pada cara perangkat keras GPIO menginterpretasikan level tegangan masukan dan tingkat kuantisasi datanya. Ketika GPIO diakses sebagai ***digital input*** (pembacaan tombol), pin hanya menggunakan pembanding ambang batas tegangan yang memetakan seluruh rentang tegangan kerja (0–3.3 V) ke dalam **dua kondisi biner (1-bit)**, yakni *LOW* (0) atau *HIGH* (1). Sebaliknya, apabila salah satu tombol digantikan dengan rangkaian pembagi tegangan LDR yang dibaca melalui ***analog input*** **(ADC)**, periferal ADC mencuplik tegangan kontinu dan mengkuantisasinya ke dalam 2^n tingkatan diskrit sesuai resolusi bitnya. Pada ESP32 dengan ADC beresolusi **12-bit**, rentang tegangan analog 0–3.3 V dipetakan menjadi $2^{12} = 4096$ **nilai digital berbeda** (0 hingga 4095, dengan resolusi sekitar 0.8 mV per langkah).

Oleh sebab itu, ADC mampu merepresentasikan banyak tingkatan kondisi intensitas cahaya secara presisi dan kontinu, tidak terbatas hanya pada dua keadaan diskrit *HIGH* atau *LOW*.

4 Kesimpulan

Berdasarkan seluruh rangkaian percobaan pada Praktikum Modul 1, dapat disimpulkan bahwa ekosistem PlatformIO pada Visual Studio Code sangat efektif memfasilitasi pengembangan lintas arsitektur (STM32 dan ESP32) serta lintas framework (Arduino dan ESP-IDF) dalam satu lingkungan terpadu. Praktikum ini menyoroti perbedaan mendasar kedua papan, di mana STM32 Blackpill memerlukan pemrograman eksternal via ST-Link (protokol SWD) dengan karakteristik LED *active-low*, sedangkan ESP32 DevKit v1 dapat diprogram langsung melalui USB menggunakan arsitektur *native* ESP-IDF berbasis FreeRTOS. Penguasaan konsep dasar GPIO digital juga terbukti krusial, mulai dari penerapan konfigurasi *pull-up* dan *pull-down* (baik eksternal maupun internal) untuk mencegah kondisi *floating*, hingga implementasi algoritma *software debouncing* berbasis **millis()** dengan jeda optimal 50 ms untuk meredam getaran mekanik sakelar. Lebih lanjut, pemanfaatan fitur ADC 12-bit pada ESP32 (GPIO 34) memperluas kapabilitas sistem tidak hanya pada pemrosesan logika diskrit, tetapi juga dalam mengkuantisasi besaran fisis kontinu dari sensor analog seperti LDR secara presisi menjadi 4096 tingkatan nilai digital.

5 Lampiran

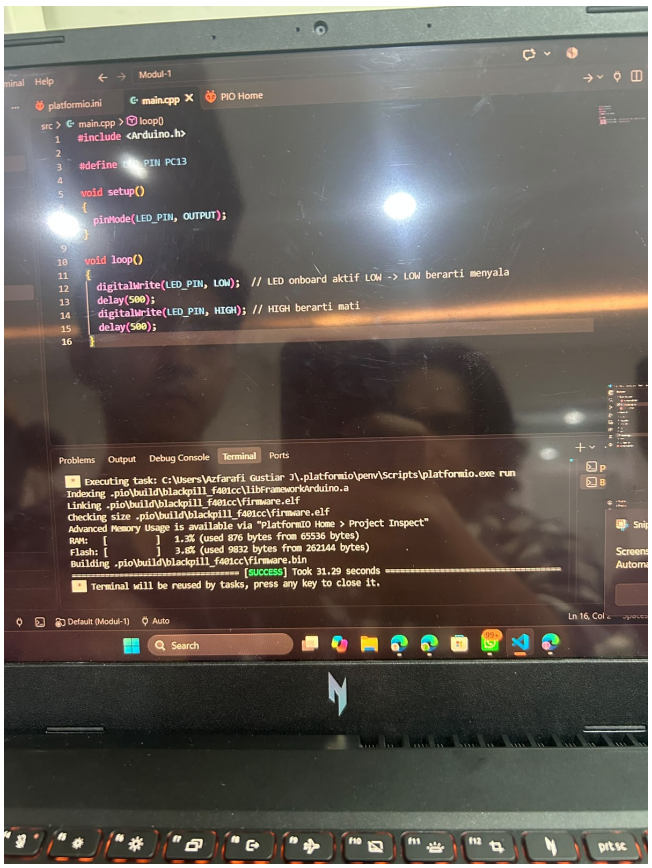
5.1 Dokumentasi saat praktikum



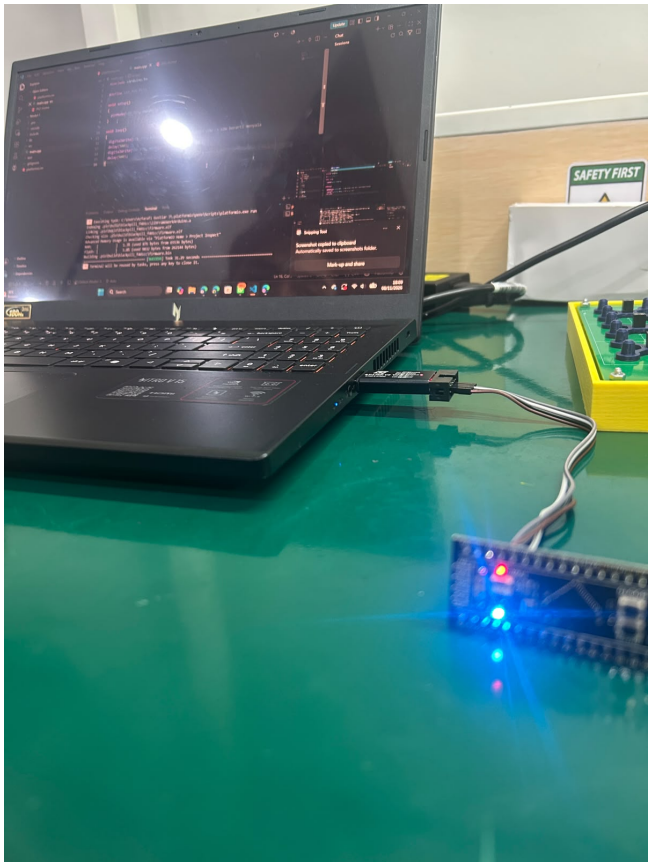
Gambar 2: Alat yang digunakan pada praktikum



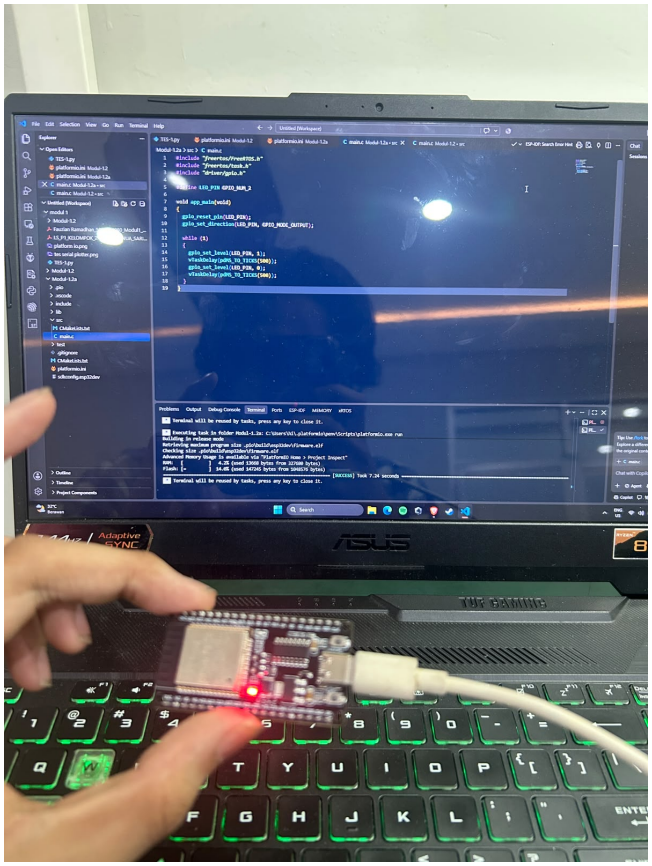
Gambar 3: Bukti praktikan mengikuti praktikum



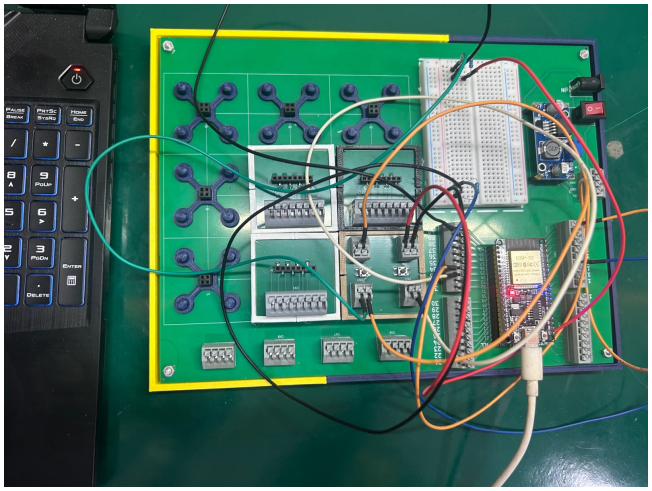
Gambar 4: Mengbuild dan upload platformio dan main ke esp32



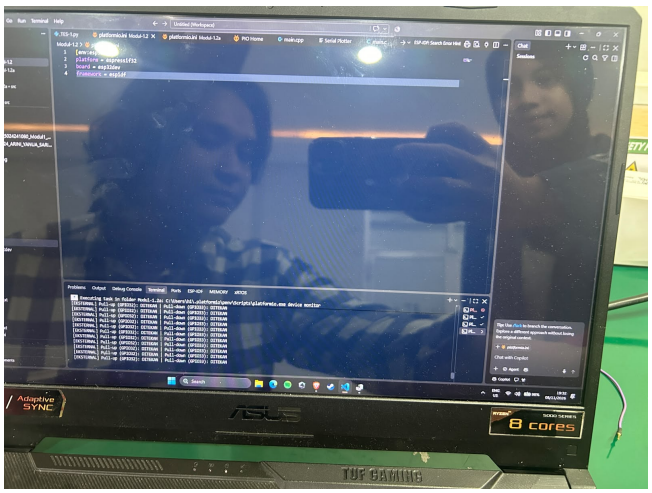
Gambar 5: Hasil Percobaan 1



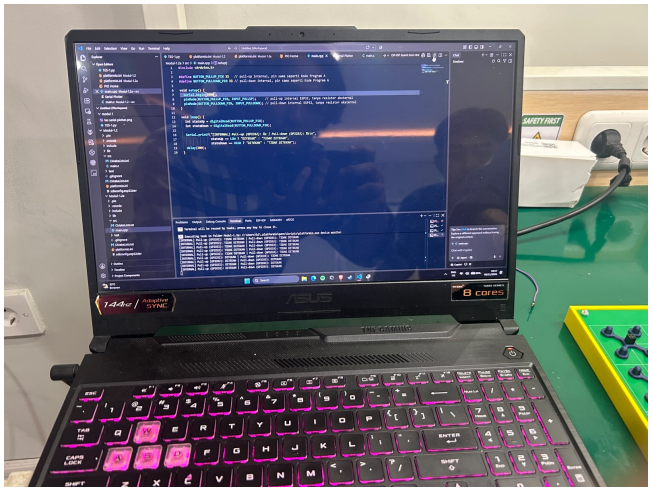
Gambar 6: Hasil percobaan 2



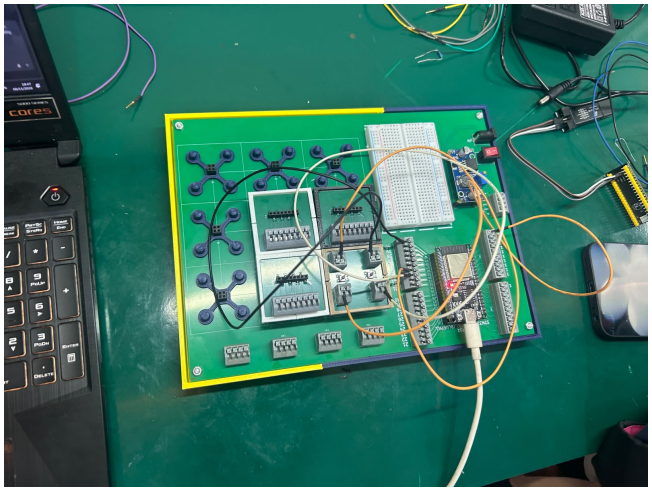
Gambar 7: Rangkaian percobaan 3a



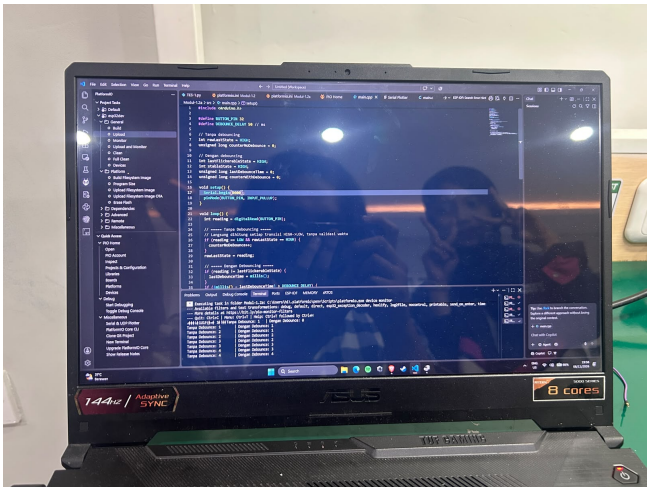
Gambar 8: Hasil percobaan 3a



Gambar 9: Hasil percobaan 3b



Gambar 10: Rangkaian percobaan 4



Gambar 11: Hasil percobaan 4a